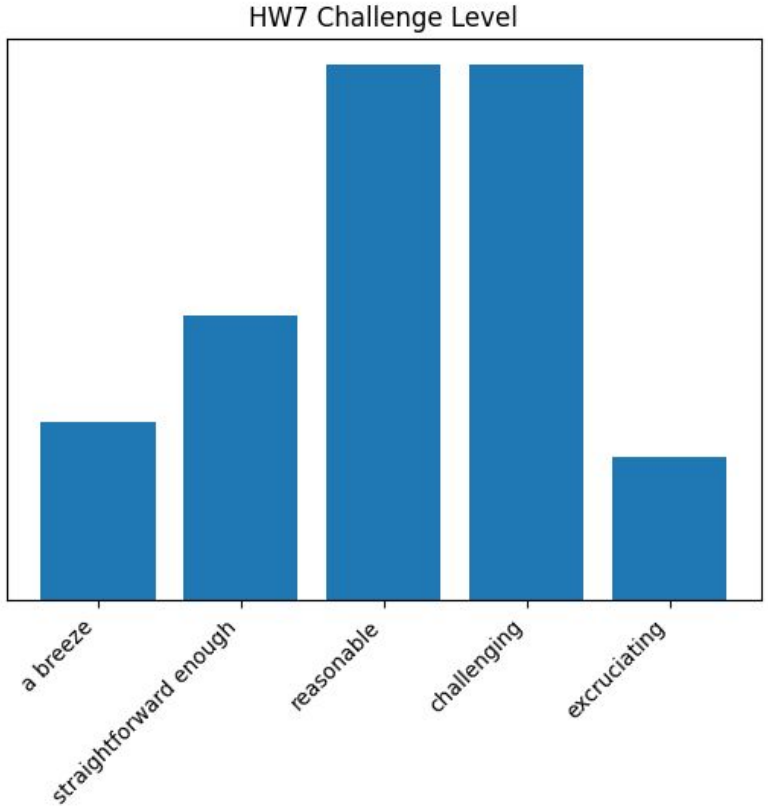
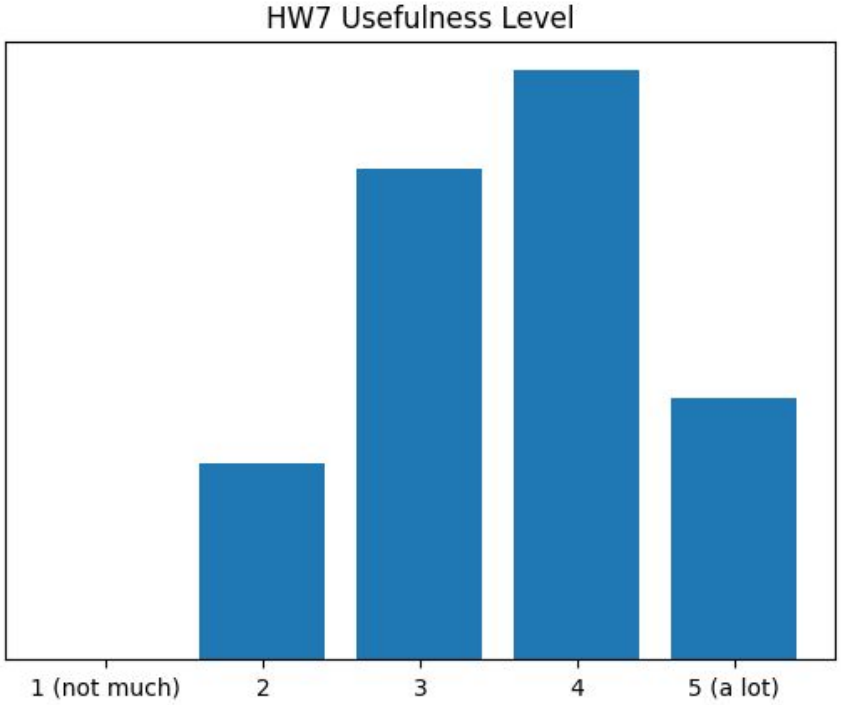
CSE 493L Section 9

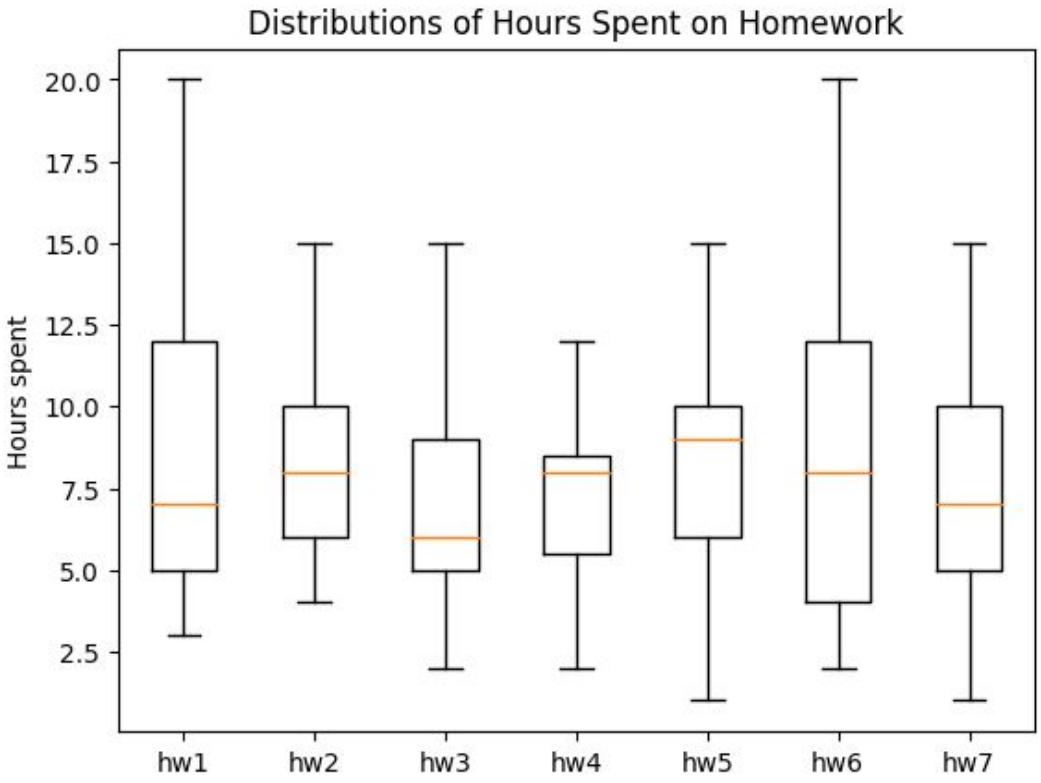
Async Server

Looking Back on Homework 7



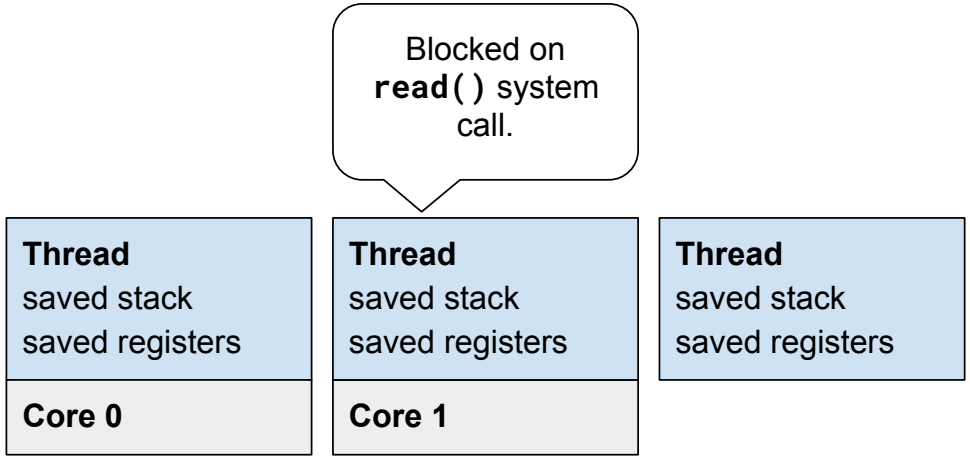


Looking Back on Homework 7



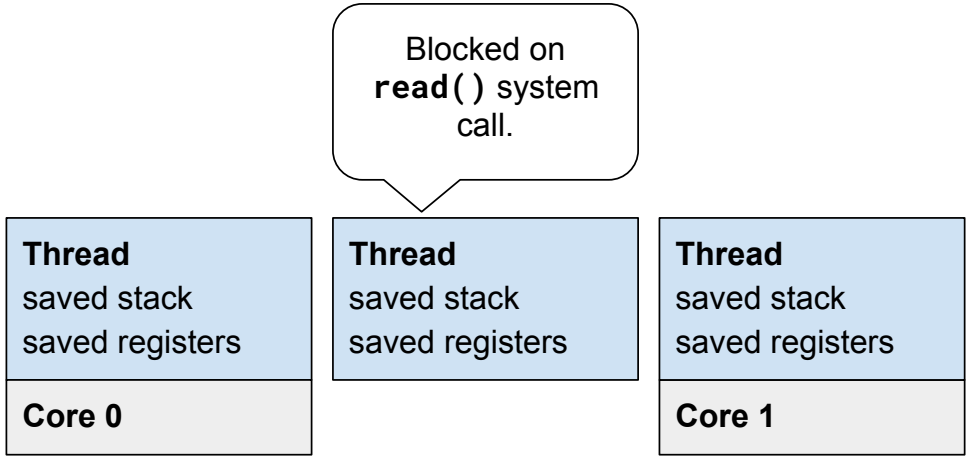
OS Threads

- OS schedules threads onto cores.
- When a thread blocks, the core switches to running another thread.



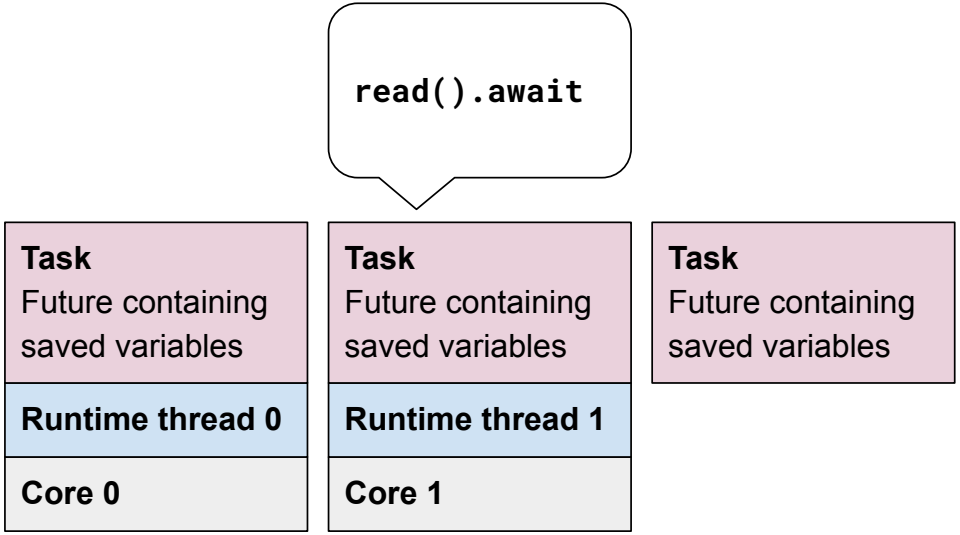
OS Threads

- OS schedules threads onto cores.
- When a thread blocks, the core switches to running another thread.



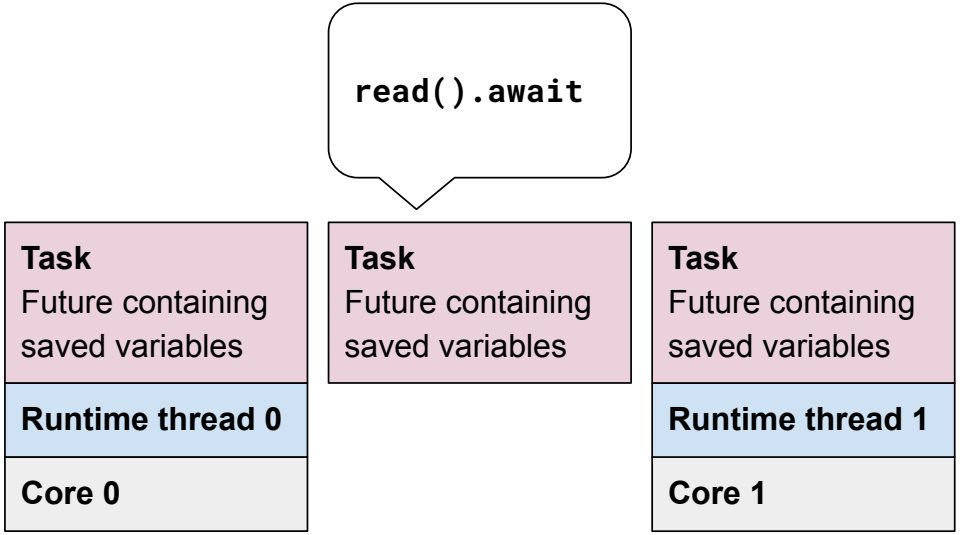
Rust Async Tasks

- Tasks scheduled onto runtime threads.
- When a task awaits, the thread switches to running another task.



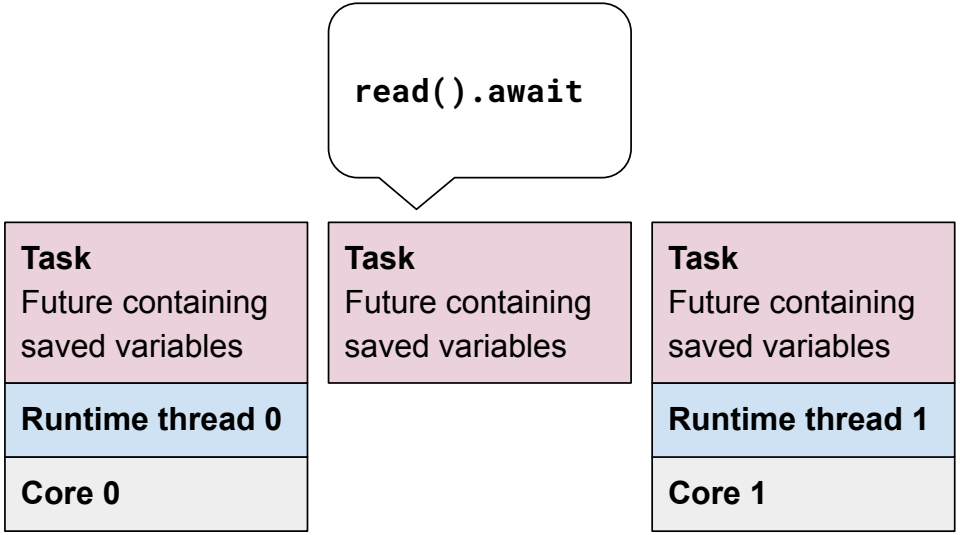
Rust Async Tasks

- Tasks scheduled onto runtime threads.
- When a task awaits, the thread switches to running another task.



Rust Async Tasks

```
// Pseudocode
fn runtime() {
  loop {
      let future = next_ready_future();
      future.poll();
  }
}
```



	Thread	Task (Future)
How is it scheduled?		
How does it wait for network?		
How are yields implemented?		
Preemption?		
How many can be spawned?		

	Thread	Task (Future)
How is it scheduled?	OS restores saved registers and changes stack pointer.	Runtime calls `future.poll()`.
How does it wait for network?		
How are yields implemented?		
Preemption?		
How many can be spawned?		

	Thread	Task (Future)
How is it scheduled?	OS restores saved registers and changes stack pointer.	Runtime calls **`future.poll()`**.
How does it wait for network?	Blocking system call like **`read()`**.	Calls Tokio function that internally makes a non-blocking system call and returns a **`future`**.
How are yields implemented?		
Preemption?		
How many can be spawned?		

	Thread	Task (Future)
How is it scheduled?	OS restores saved registers and changes stack pointer.	Runtime calls **`future.poll()`**.
How does it wait for network?	Blocking system call like **`read()`**.	Calls Tokio function that internally makes a non-blocking system call and returns a **`future`**.
How are yields implemented?	Saves all registers and changes stack pointer.	**`future.poll()`** puts in-use variables in its state machine, and returns to the runtime.
Preemption?		
How many can be spawned?		

	Thread	Task (Future)
How is it scheduled?	OS restores saved registers and changes stack pointer.	Runtime calls **`future.poll()`**.
How does it wait for network?	Blocking system call like **`read()`**.	Calls Tokio function that internally makes a non-blocking system call and returns a **`future`**.
How are yields implemented?	Saves all registers and changes stack pointer.	**`future.poll()`** puts in-use variables in its state machine, and returns to the runtime.
Preemption?	Non-cooperative (preemptable). Can be interrupted whenever.	Cooperative (non-preemptable). Only yields on **`await`**.
How many can be spawned?		

	Thread	Task (Future)
How is it scheduled?	OS restores saved registers and changes stack pointer.	Runtime calls **`future.poll()`**.
How does it wait for network?	Blocking system call like **`read()`**.	Calls Tokio function that internally makes a non-blocking system call and returns a **`future`**.
How are yields implemented?	Saves all registers and changes stack pointer.	**`future.poll()`** puts in-use variables in its state machine, and returns to the runtime.
Preemption?	Non-cooperative (preemptable). Can be interrupted whenever.	Cooperative (non-preemptable). Only yields on **`await`**.
How many can be spawned?	Up to ~10,000s (out of memory)	Millions

Async Usage

Compiler transforms an async function into a normal function that immediately returns:

```
async fn func() -> i32 {
    fetch().await
}
```

```
fn func() -> impl Future<Output = i32> {
    return CompilerGeneratedStateMachine { /* ... */ };
}
```

To run a future to completion (yielding if necessary):

```
let future = func();
let out: i32 = future.await;
```

To spawn a future in the background:

```
let future = func();
let handle = tokio::spawn(future);
```

Blocking

```rust
async fn fetch() -> i32 {
    blocking_read(); // BAD

    tokio_read().await // GOOD
}
```

- Don’t block inside async functions.
  - That would block the entire runtime thread, preventing it from switching to other tasks.
- Instead, await on non-blocking Tokio alternatives.
  - These make a non-blocking system call and immediately return a future.
  - **`future.await`** yields to the runtime.
- Exception: you can typically use **`std::sync::Mutex`**
  - Just don't block or await while holding it.
  - Otherwise use a Tokio mutex (this integrates with the runtime).

Cancellation

```rust
let handle = thread::spawn(func);
handle.abort(); // impossible

let handle = tokio::spawn(async_func());
handle.abort(); // possible
```

- It’s impossible to cancel a running OS thread.
  - There’s no reliable way to run destruct its variables after interrupting it.
- It is possible to cancel a running async task.
  - The runtime simply drops its **`Future`** struct. This gracefully destructs all its fields.
  - Only works if the **`Future`** isn’t blocked or doing long-running computation
    - Only do such things in threads!

Cancellation Enables Timeouts!

```rust
let result = tokio::time::timeout(DURATION, async_func()).await;
match result {
   Ok(returned) => println!("Result: {returned}"),
   Err(_elapsed_error) => println!("Timeout"),
}
```

- Drops the future if it doesn’t finish by the timeout.
- Works on all async functions!

Concise IO Timeouts

```
// Try to read a message from network
let result = timeout(DURATION, read_msg(&mut stream)).await;

let msg: ClientMsg = match result {

};
println!("received {msg}");
```

```
Result<ClientMsg, io::Error>
```

```
Result<_, error::Elapsed>
```

What goes here?

Concise IO Timeouts

```
// Try to read a message from network
let result = timeout(DURATION, read_msg(&mut stream)).await;

let msg: ClientMsg = match result {
   Ok(Ok(msg)) => msg,




};
println!("received {msg}");
```

```
Result<ClientMsg, io::Error>
```

```
Result<_, error::Elapsed>
```

Concise IO Timeouts

```
// Try to read a message from network
let result = timeout(DURATION, read_msg(&mut stream)).await;

let msg: ClientMsg = match result {
   Ok(Ok(msg)) => msg,
   Ok(Err(_io_error)) => return,




};
println!("received {msg}");
```

```
Result<ClientMsg, io::Error>
```

```
Result<_, error::Elapsed>
```

Concise IO Timeouts

```
// Try to read a message from network
let result = timeout(DURATION, read_msg(&mut stream)).await;

let msg: ClientMsg = match result {
   Ok(Ok(msg)) => msg,
   Ok(Err(_io_error)) => return,
   Err(_elapsed_error) => {
       _ = write_msg(ServerMsg::ErrorTimeout, &mut stream).await;
       return;
   }
};
println!("received {msg}");
```

```
Result<ClientMsg, io::Error>
```

```
Result<_, error::Elapsed>
```

Cancellation: `select!` (“race”)!

```
tokio::select! {
    returned = func1() => {
        println!("func_1 finished first: {returned}");
    }
    returned = func2() => {
        println!("func_2 finished first: {returned}");
    }
}
```

Starts multiple futures. When one finishes, cancels the others.

Channels

```rust
// Pipe with buffer of size 8
let (tx, mut rx) = mpsc::channel(8);

tokio::spawn(async move {
   let received = rx.recv().await.expect("no more senders");
   assert_eq!(received, 5);
});
tx.send(5).await.expect("no more receivers");
```

Just like the channels you implemented in homework 1!

Common Pattern: Split

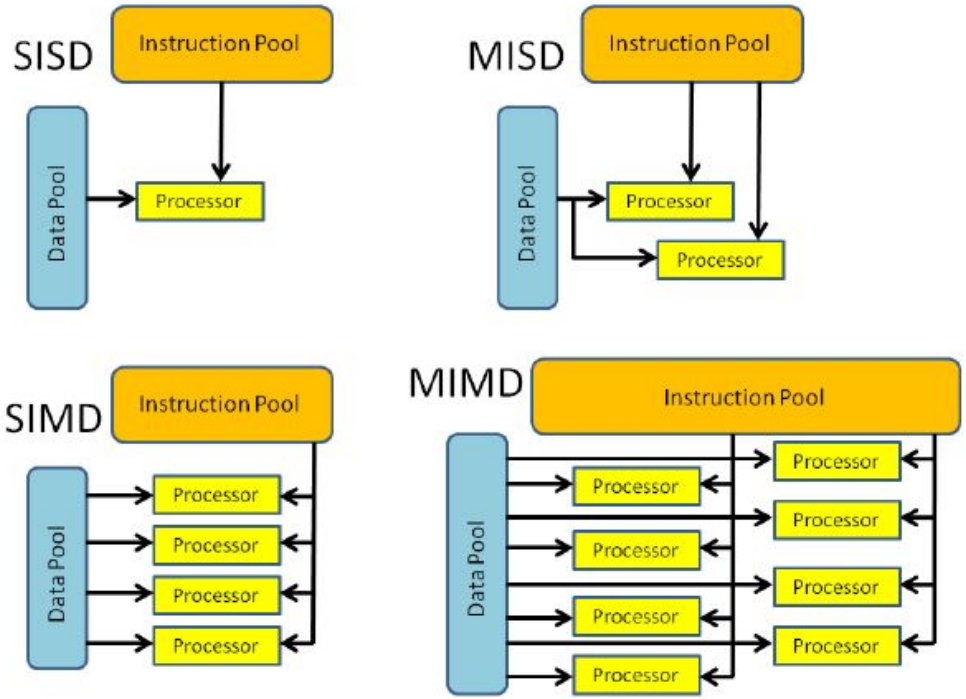
```rust
let (read_half, write_half) = tokio::io::split(connection);
let (tx, rx) = mpsc::channel(8);
let read_handle = tokio::spawn(read_task(tx, read_half));
let write_handle = tokio::spawn(write_task(rx, write_half));
```

- Spawn 2 tasks: for read and write.
- If **`read_task`** (or anyone else) wants something to be written:
  - send that to **`write_task`** over a channel.

Async in Other Languages

- **Javascript**: Called **Promise** instead of **Future**. A returned **Promise** starts running immediately, unlike lazy Rust **Futures**.
- **Go**: Each “goroutine” has its own stack, resized by garbage collector. Implicit yield/await points are inserted by the compiler.
- **C**: Handwritten event loop with non-blocking system calls and state machines.

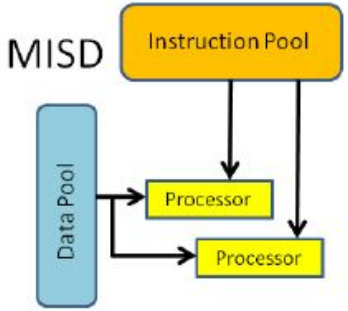
Flynn's Taxonomy

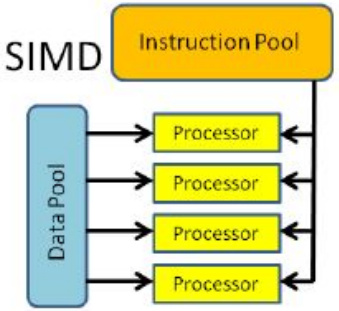


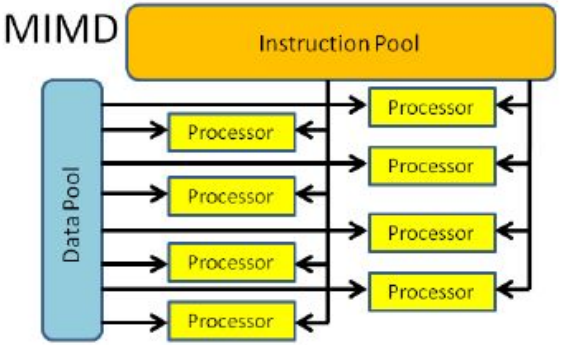
Flynn's Taxonomy

SISD

At any given time step, the **single processor** pulls a **single instruction**, which acts on a **single data stream**







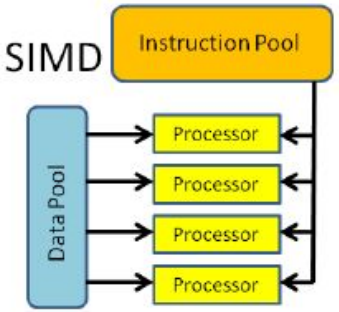
Flynn's Taxonomy

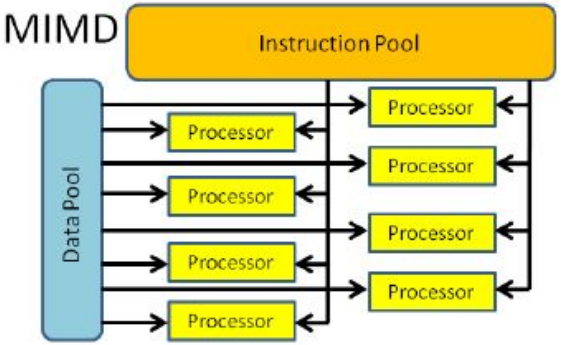
SISD

At any given time step, the **single processor** pulls a **single instruction**, which acts on a **single data stream**

MISD

At any given time step, each of the **multiple processors** pull **different instructions**, which acts on a **single data stream**





Flynn's Taxonomy

SISD

At any given time step, the **single processor** pulls a **single instruction**, which acts on a **single data stream**

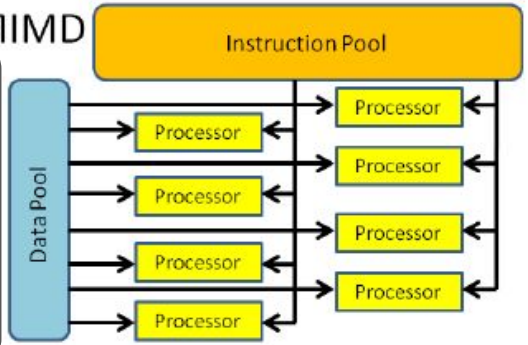
MISD

At any given time step, each of the **multiple processors** pull **different instructions**, which acts on a **single data stream**

SIMD

At any given time step, each of the **multiple processors** pulls a **single instruction**, which acts on **multiple data stream**

MIMD



Flynn’s Taxonomy

SISD

At any given time step, the **single processor** pulls a **single instruction**, which acts on a **single data stream**

MISD

At any given time step, each of the **multiple processors** pull **different instructions**, which acts on a **single data stream**

SIMD

At any given time step, each of the **multiple processors** pulls a **single instruction**, which acts on **multiple data stream**

MIMD

At any given time step, each of the **multiple processors** pull **different instructions**, which act on **multiple data stream**

Flynn's Taxonomy

Uniprocessors, Old PCs, Some Embedded Processors

SISD

At any given time step, the **single processor** pulls a **single instruction**, which acts on a **single data stream**

MISD

At any given time step, each of the **multiple processors** pull **different instructions**, which acts on a **single data stream**

Rare! (apparently used in some space shuttles, mostly for fault tolerance)

Array Processors (GPUs, SIMT), Pipeline Processors

SIMD

At any given time step, each of the **multiple processors** pulls a **single instruction**, which acts on **multiple data stream**

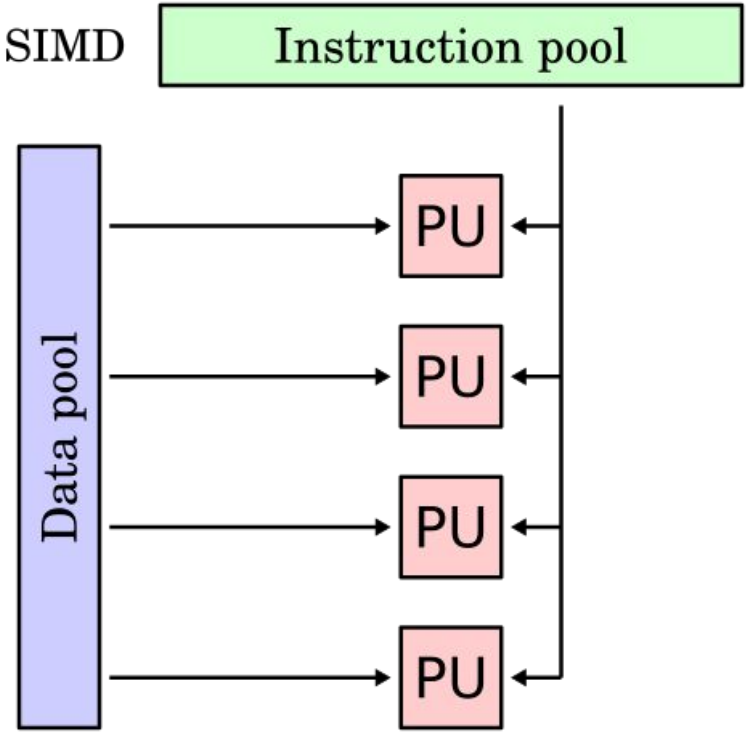
MIMD

At any given time step, each of the **multiple processors** pull **different instructions**, which act on **multiple data stream**

Distributed Systems, Multi-core Superscalar Processors

Flynn’s Taxonomy

GPU architecture limits what kinds of programs they can execute quickly

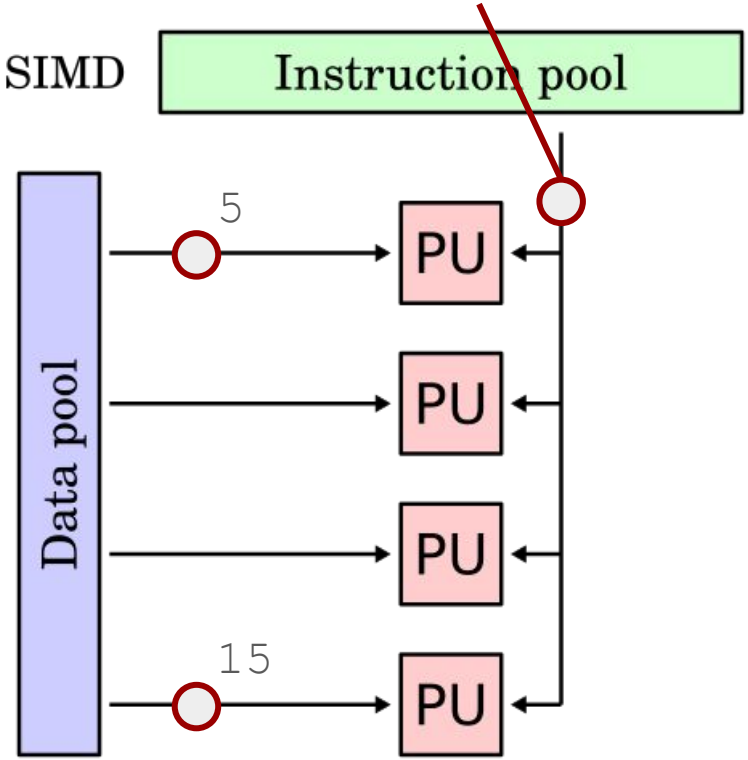


Flynn's Taxonomy

GPU architecture limits what kinds of programs they can execute quickly

* this is a huge simplification, in reality "instructions" really are instructions

```
if x < 10 {x++} else {x--}
```



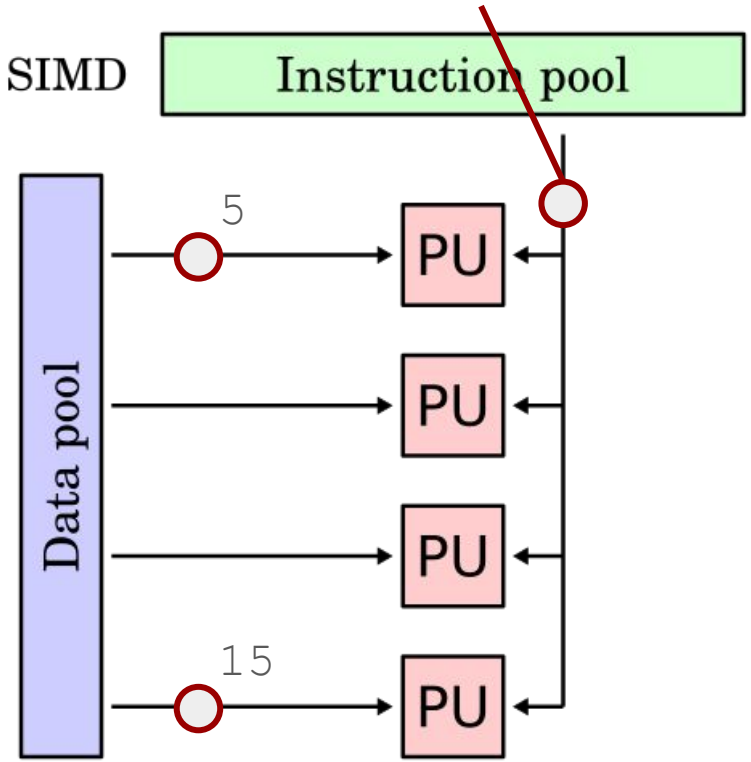
Flynn's Taxonomy

* this is a huge simplification, in reality "instructions" really are instructions

```
if x < 10 {x++} else {x--}
```

GPU architecture limits what kinds of programs they can execute quickly

- GPU can't add and subtract with a single instruction!
- GPU has to spend time *resolving all branches* so the proper operation gets run.



"Branchless programming" is important, you'll also hear about "vectorized operations"

Flynn’s Taxonomy - SIMD vs. SIMT

You’ll sometimes hear SIMT (multi-thread) in the context of GPUs

- From a programmer's perspective, they are basically the same model
- SIMT is a slightly “higher-level abstraction”
  - Threads are issued the same instruction, and operate in lock step with one another
    - Each thread has its own PC
  - Threads can be “masked off”, resulting in a no-op on a particular data stream
  - Threads maintain more state, owning fast access register files
  - SIMT allows memory coalescing, one thread fetches memory potentially used by all threads
  - SIMT allows for deeper compiler optimizations…

What Architecture Should We Use?

You are making image filter software, which applies a specific transformation to each pixel value

What Architecture Should We Use?

You are making image filter software, which applies a specific transformation to each pixel value

SIMD

What Architecture Should We Use?

You are making software for a submarine, which takes a sensor reading and runs multiple functions over the result, which then determine a bunch of controls

What Architecture Should We Use?

You are making software for a submarine, which takes a sensor reading and runs multiple functions over the result, which then determine a bunch of controls

MISD

What Architecture Should We Use?

You are making a video game engine, which keeps track of entity positions and runs physics simulations

What Architecture Should We Use?

You are making a video game engine, which keeps track of entity positions and runs physics simulations

MIMD

What Architecture Should We Use?

You are making an ecommerce web server, which responds to multiple different kinds of requests to let users shop

What Architecture Should We Use?

You are making an ecommerce web server, which responds to multiple different kinds of requests to let users shop

MIMD

What Architecture Should We Use?

You are multiplying a bunch of matrices together

What Architecture Should We Use?

You are multiplying a bunch of matrices together

SIMD

What Architecture Should We Use?

You are making an embedded elevator control module

What Architecture Should We Use?

You are making an embedded elevator control module

SISD

What Architecture Should We Use?

You have a camera on every intersection of Seattle. These cameras provide real-time footage of traffic, that you need to use to coordinate traffic lights to allow for maximum traffic flow.

What Architecture Should We Use?

You have a camera on every intersection of Seattle. These cameras provide real-time footage of traffic, that you need to use to coordinate traffic lights to allow for maximum traffic flow.

It depends… probably some combination of MIMD and SIMD